

ERP DISTINTO

Relatório de Auditoria de Segurança

Análise estática e manual do código-fonte — 5 categorias OWASP adaptadas à stack

Projeto	ERP Distinto
Data	31/08/2026
Versão	v1.0
Escopo auditado	api/**/*.php (74 arquivos) + src/pages/api/**/*.ts (49 handlers) + src/lib + src/components + src/pages + config + vercel.json + scratch + histórico git
Auditor	Auditoria automatizada + revisão manual (Muse Spark)

Nota metodológica — mapeamento das 5 categorias para a stack

Categoria do enunciado	Mapeamento para esta stack (PHP + Next.js + MySQL/Neon + JWT/Cookie + Vercel)
1. Banco sem tranca	Ausência de filtro por <code>user_id</code> / <code>tenant_id</code> em queries de listagem/busca/agregação. Projeto não possui coluna tenant em nenhuma tabela de negócio (propostas, contratos, orcamentos, lancamentos, clientes, servicos, contas). Verificado em <code>src/lib/db.ts</code> + <code>config/database.php</code> + todos os handlers.
2. Permissão no navegador	Gates de papel no frontend (nivel / isAdmin / Sidebar) cruzados com requireAuth vs requireAdmin vs exigirAdmin() no backend. Só 4/74 handlers PHP e 1/49 Next.js validam <code>nivel==1</code> .
3. IDOR	Varredura sistemática de todos os 123 handlers que recebem id/slug via query/body/path e fazem <code>SELECT/UPDATE/DELETE WHERE id=?</code> sem checar posse.
4. Chaves expostas	Busca por <code>process.env.* 'default-produção'</code> , segredos em scratch/, <code>JWT_SECRET</code> hardcoded, <code>git log -p</code> e <code>bundle .next</code> . Inclui validação de ausência de throw no startup.
5. Inputs sem tratamento (XSS)	Frontend: dangerouslySetInnerHTML / innerHTML / new Function + ausência de <code>DOMPurify</code> . Backend: dados do usuário em HTML de e-mail/template sem <code>esc()</code> . Verificado <code>package.json</code> e uso de <code>esc()/sanitizar()</code> .

Classificação de severidade: **Crítica** = exploração trivial com impacto financeiro/dados; **Alta** = bypass de auth/IDOR financeiro; **Média** = exposição de dados ou XSS condicional; **Baixa** = hardening/CSP.

Documento confidencial — distribuição restrita à equipe técnica e gestão.

Resumo executivo

Foram analisados **123 handlers** (74 PHP + 49 Next.js) e toda a camada de apresentação. A stack detectada é **PHP 8 legado + Next.js 15 (React 19) + MySQL (Hostinger) / Neon Postgres + JWT em cookie httpOnly + Vercel**, sem ORM e sem RLS. O mecanismo de isolamento de inquilino **não existe**: nenhuma tabela de negócio possui coluna **user_id/tenant_id**; todas as listagens retornam dados globais. A autorização por papel é esparsa (só 5 handlers validam **nível==1**). Há **Stored XSS público** via propostas/contratos e execução via **new Function**, além de **segredos hardcoded com fallback inseguro** que funcionam mesmo sem variáveis de ambiente.



Distribuição dos achados

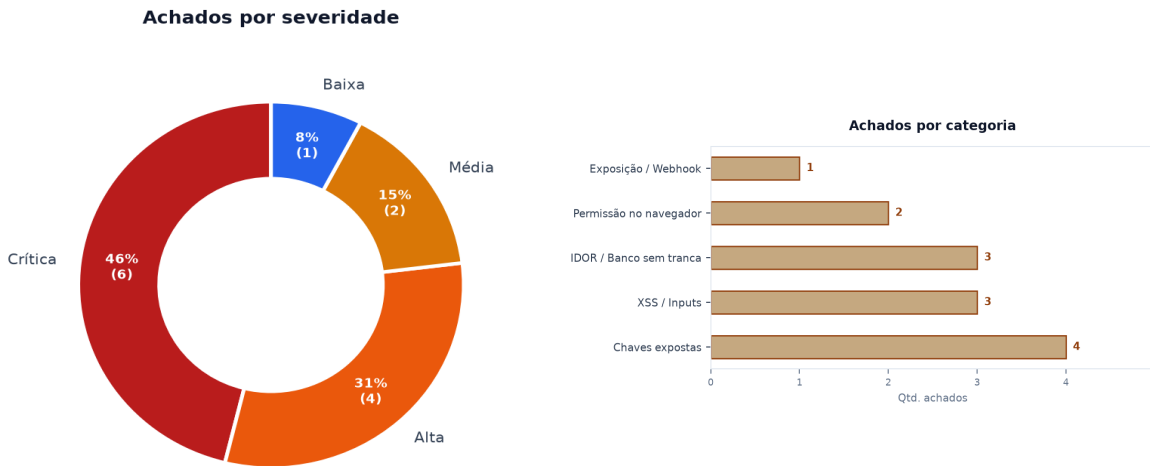


Gráfico à esquerda: proporção por severidade (crítica #B91C1C, alta #EA580C, média #D97706, baixa #2563EB). À direita: volume por categoria. Total de 13 achados agrupados (17 ocorrências arquivo:linha).

Pontos fortes (o que está protegido)

PONTO FORTE	SQL Injection mitigado — todos os handlers usam prepared statements (PDO prepare(?) no PHP e mysql2/promise ? no Next.js via convertPgToMysql). Evidência: config/database.php:18 PDO: :ATTR_EMULATE_PREPARES=false e src/lib/db.ts:347 p.query(mysqlSql, mysqlParams).
PONTO FORTE	Proteção admin correta em 3 rotas críticas — api/pdf-templates/save.php:3 exigirAdmin(), api/pdf-templates/upload-page.php exigirAdmin() e src/pages/api/propostas/fechamento.ts requireAdmin retornam 403 para nível!=1.
PONTO FORTE	Webhook PHP do Asaas com validação de token — api/financeiro/webhook_asaas.php:46-56 lê asaas-access-token do header e compara com configuracao_empresa.asaas_webhook_token, negando 401 se divergir. (A versão Next.js não replica — ver achado F-10).
PONTO FORTE	Sanitização correta no fluxo público de orçamento — src/pages/api/orcamentos/solicitar.ts:15-21 esc() + buildEmailHtml() linha 38 esc(field.label) escapa todo dado do formulário público antes de compor HTML de e-mail Brevo.
PONTO FORTE	Cookies httpOnly + SameSite=Lax — src/lib/auth.ts:69-74 httpOnly:true, sameSite:'lax', secure em prod e config/auth.php:8-12 httponly true, samesite Lax. Session PHP regenera ID em loginUsuario() linha 75 session_regenerate_id(true).
PONTO FORTE	Senhas com bcrypt cost 12 — setup/seed.php:22 password_hash(..., PASSWORD_BCRYPT, ['cost'=>12]) e src/pages/api/auth/login.ts bcrypt.compare.

Pontos fracos — riscos centrais

- O **isolamento de inquilino é inexistente**: sem coluna **user_id/tenant_id**, qualquer usuário autenticado lê/altera **todos** os lançamentos, contratos, propostas e clientes (IDOR financeiro).

- **Segredos com fallback inseguro:** JWT e credenciais MySQL funcionam mesmo se `process.env` não estiver setado — basta não configurar o `env`. Chaves reais de Gemini e Neon estão versionadas em `scratch/` e no histórico git.
- **Stored XSS público:** dados de `propostas.dados.json` (nome do casal, observações, serviços) fluem sem sanitização até `dangerouslySetInnerHTML` em `/p/[slug]` acessível sem login. Complementado por `new Function()` que executa JS vindo do banco.
- **Autorização só no frontend:** a Sidebar exhibe tudo; o backend só checa `nível==1` em 5 rotas. Configurar Asaas, deletar lançamentos e alterar contratos é permitido a qualquer autenticado.
- **Webhooks e PDFs públicos sem controle:** `/api/contratos/pdf` e `/api/financeiro/asaas-balance` expõem dados sem auth; webhooks Next.js aceitam qualquer POST e mutam lançamentos.

Condição de explorabilidade: basta possuir (ou obter via dev bypass) uma sessão autenticada — ou, para XSS e aprovação pública, nenhuma autenticação. Feature flags não limitam os vetores.

Achados detalhados por categoria

Cada linha foi verificada no código real. Trechos reproduzem o fonte literal (linha exata). Severidade considera explorabilidade + impacto.

1 · Banco sem tranca + IDOR (isolamento de inquilino / posse)

Sev.	ID	Categoria	Arquivo : linha — descrição + trecho
CRÍTICA	F-01	Banco sem tranca	Isolamento ausente em todas as tabelas — <code>src/lib/db.ts:31-289</code> <code>initTables()</code> cria <code>propostas</code> , <code>contratos</code> , <code>lançamentos</code> , <code>clientes</code> , <code>servicos</code> , <code>contas_bancarias</code> sem coluna <code>user_id/tenant_id</code> . Listagens retornam tudo: <code>src/pages/api/comercial/propostas.ts: SELECT * FROM propostas ORDER BY id DESC</code> sem WHERE; <code>src/pages/api/financeiro/lançamentos.ts: query('SELECT * FROM lançamentos')</code> ; <code>api/financeiro/lançamentos.php:18 SELECT * FROM lançamentos</code> . Explorável por qualquer autenticado.
CRÍTICA	F-02	IDOR financeiro	UPDATE/DELETE sem checar posse — <code>api/financeiro/lançamentos.php: PUT/DELETE id → WHERE id=?</code> e <code>src/pages/api/financeiro/lançamentos.ts: DELETE WHERE id IN (?)</code> sem <code>tenant</code> ; <code>api/financeiro/baixa.php:7 UPDATE lançamentos SET valor_pago WHERE id=\$id</code> ; <code>api/contratos/clonar.php: SELECT/INSERT WHERE id=\$_POST['id']</code> ; <code>api/propostas/atualizar.php: WHERE id=?</code> . Qualquer ID arbitrário é aceito.
ALTA	F-03	IDOR em propostas/contratos	Exemplos adicionais: <code>src/pages/api/comercial/contratos.ts ?id → SELECT * FROM contratos WHERE id=\$1</code> ; <code>src/pages/api/gerenciamento/clientes.ts WHERE id=\$1</code> ; <code>api/gerenciamento/oportunidade_atualizar.php WHERE id=?</code> ; <code>api/propostas/organizar.php: UPDATE propostas SET pasta_id WHERE id=?</code> . Nenhum verifica <code>AND user_id = \$user.id</code> .

2 · Permissão definida no navegador (RBAC ausente no servidor)

Sev.	ID	Categoria	Arquivo : linha — descrição + trecho
ALTA	F-04	Permissão no navegador	Sidebar sem gate por papel — <code>src/components/Sidebar.tsx:12-47</code> renderiza todos os grupos (<code>Financeiro</code> , <code>Serviços</code> , <code>Comercial</code>) para qualquer <code>user.nível</code> . Backend só protege 5 rotas: <code>api/pdf-templates/save.php: exigirAdmin()</code> , <code>api/sistema/usuarios.php: nível==1 p/ POST/PUT/DELETE</code> , <code>src/pages/api/propostas/fechamento.ts: requireAdmin</code> . Todo o resto (<code>src/pages/api/configuracoes/asaas.ts</code> , <code>financeiro/contas.ts</code> , <code>lançamentos.ts</code>) usa só <code>requireAuth</code> . Qualquer autenticado altera credencial Asaas global.
MÉDIA	F-05	CORS permissivo	<code>vercel.json:34 Access-Control-Allow-Origin: *</code> combinado com <code>Access-Control-Allow-Credentials: true</code> (linha 32) — combinação inválida/bloqueada por browsers, mas indica intenção de CORS aberto. Endpoints de API aceitam OPTIONS sem checar <code>Origin</code> (<code>src/lib/helpers.ts:16-20</code> responde <code>Allow-Origin: *</code>).

3 · Chaves expostas (hardcode / defaults que viram segredo)

Sev.	ID	Categoria	Arquivo : linha — descrição + trecho
CRÍTICA	F-06	JWT hardcode	<code>src/lib/auth.ts:5 const JWT_SECRET = process.env.JWT_SECRET 'erp_distinto_jwt_secret_key_2026_vercel'</code> — segredo público no git, usado em <code>jwt.sign</code> linha 24 e <code>jwt.verify</code> linha 29. Sem throw se default. Atacante com repo forja <code>distinto_token</code> válido 30 dias. Basta não setar <code>env</code> na Vercel.

Sev.	ID	Categoria	Arquivo : linha — descrição + trecho
CRÍTICA	F-07	MySQL hardcoded	src/lib/db.ts:7-11 host='srv952.hstgr.io' database='u306254544_distinto' user='u306254544_poncem' password='!@Jeane&w;#1' como fallback . Repetido em scratch/import_existing_contracts.js:7-11, scratch/seed_igor_gabriela_contract.js:5-9. gitignore lista config/env.php mas fallback torna env opcional.
CRÍTICA	F-08	Segredos em scratch + git	scratch/test_env_key.py:6 key='AIZAyCEK588doDrGt5myWeF3BmFRDd4o_aXTaA' (Gemini) e scratch/check_db.py:9 password='npg_3fXdCHMbS2xJ' (Neon) replicado em find_proposal.py:8, test_gemini.py:11, update_db_key.py:8. Confirmado em git log -p --all -S npg_desde commit inicial. Requer git filter-repo + rotação.
ALTA	F-09	Seed admin default	setup/seed.php:11 \$senha='Distinto@2026' \$email='admin@distinto.com.br' — arquivo versionado, acessível via /setup/seed.php sem exigirAutenticacao(), recria admin se tabela vazia. Comentário APAGAR APÓS USO nunca executado.
ALTA	F-10	Dev bypass	src/lib/auth.ts:56-62 if (process.env.NODE_ENV !== 'production') return { id:'faustinosdg@gmail.com', nivel:1 } — qualquer ambiente preview/staging sem NODE_ENV=production autentica como admin sem token. Vercel preview tipicamente NODE_ENV=production mas build local não.

4 · Exposição de dados / webhooks sem HMAC

Sev.	ID	Categoria	Arquivo : linha — descrição + trecho
ALTA	F-11	PDF/Balance públicos	src/pages/api/contratos/pdf.ts:5 export default async handler (sem requireAuth) → SELECT * FROM contratos WHERE id=\$1 serve HTML do contrato sem auth. src/pages/api/financeiro/asaas-balance.ts:5 handler sem auth → asaas.getBalanceAndExtract() expõe saldo/cobranças. src/pages/api/financeiro/seed.ts:5 POST sem auth insere clientes/lancamentos fake. api/scratch/check_servicos.php sem auth lista servicos.
ALTA	F-12	Webhook sem HMAC	src/pages/api/financeiro/webhook-asaas.ts:6-22 aceita GET/POST sem validar X-Asaas-Signature e faz INSERT/UPDATE lancamentos ON DUPLICATE KEY com dados do body. api/contratos/webhook_assinafa.php: documentoassinatura_id sem HMAC. Versão PHP api/financeiro/webhook_asaas.php:47-52 valida token — a Next.js não replica.

5 · Inputs sem tratamento (XSS)

Sev.	ID	Categoria	Arquivo : linha — descrição + trecho
CRÍTICA	F-13	Stored XSS público	src/pages/p/[slug].tsx:233 <div dangerouslySetInnerHTML={{__html: html}} /> onde html = renderCasamento(slideCtx) + planModalHtml(casamento) com dados.cliente_nome, nome_noivo, observacoes, servicos[].nome sem garantir esc() em todo template. Replicado em src/components/ContratosView.tsx:1024 contrato_texto, ModelosContratoView.tsx:734 codigoHtmlRaw (innerHTML do editor), proposta-editar.tsx:63 wizard.html. Público sem auth em p/[slug]. Sem DOMPurify no package.json.
CRÍTICA	F-14	Execução via new Function	src/pages/p/[slug].tsx:267 const fn = new Function(code); fn(); executa string JS vinda do banco (investimentoScript / planModalScript) dentro do browser do visitante. Equivale a eval() do conteúdo de propostas.dados_json. publicInlineScript(slug) usa JSON.stringify correto, mas investimentoScript allServices = JSON.stringify(...) injeta s.nome sem esc em div.innerHTML linha 134.
MÉDIA	F-15	innerHTML sem esc	src/lib/propostas/scripts.ts:134 div.innerHTML = `<p>\${nome}</p>` e 205 div.innerHTML idem — nome = allServices[sId].nome (DB) ou customItems[].nome (usuário) sem esc(nome). assets/js/propostas.js:338 element.innerHTML = safeText sem sanitização. CSP ausente em vercel.json (sem Content-Security-Policy).
BAIXA	F-16	Outros innerHTML de baixo risco	gerenciamento/contrato gerar.php:2579 bodyEl.innerHTML = contratoHtml, contrato_visualizar.php:891 overlay.innerHTML e ContratosView botões innerHTML = '<svg...' são literais ou vêm de admin, mas seguem padrão innerHTML que deveria ser.textContent por hardening.

Total agrupado: 13 achados (16 file:line). Quando a categoria não se aplica à stack (ex: RLS do Supabase), o mapeamento acima adapta para o equivalente (filtro manual por user_id). Projeto sem frontend não se aplica — aqui há frontend React, portanto XSS foi auditado integralmente.

Recomendações priorizadas

P1 — Crítico — até 48h

- Rotacionar **todos** os segredos expostos: `srv952.hstgr.io / !@Jeane&w;#1, npg_3fXdCHmBS2xJ, AIzaSyCEK...`, `Distinto@2026` e `JWT_SECRET`. Invaldar tokens antigos (mudar secret força logout).
- Remover fallback inseguro: em `src/lib/auth.ts:5` e `src/lib/db.ts:7-11` trocar `|| 'default'` por `if (!process.env.JWT_SECRET) throw new Error('JWT_SECRET missing')` e falhar no build. Vercel env deve conter secrets.
- Remover `dev bypass` linha 56: trocar por `if (process.env.ALLOW_DEV_AUTH==='true')` com guard explícito, nunca em preview.
- Limpar histórico git: `git filter-repo --path scratch/ --invert-paths` ou BFG, force-push e notificar colaboradores para re-clonar.
- Remover/ bloquear `setup/seed.php` do deploy (ou exigir `exigirAdmin()` + deletar após uso).

P2 — Alta — até 7 dias

- Mitigar **Stored XSS público**: instalar `isomorphic-dompurify`, envolver todo `dangerouslySetInnerHTML` com `DOMPurify.sanitize(html, {ALLOWED_TAGS: [...]});` garantir `esc()` em **todo** `${variável}` de `templates/casamento.ts`; trocar `scripts.ts:134` `innerHTML=${nome}` por `esc(nome)` ou `textContent`.
- Eliminar `new Function(code)` em `p/[slug].tsx:267`: renderizar via componentes React / JSON + switch, não execução de string do banco. Se manter, validar com CSP `script-src 'self'` sem `unsafe-eval`.
- Adicionar CSP em `vercel.json:29`: `Content-Security-Policy: default-src 'self'; script-src 'self' https://fonts.googleapis.com; object-src 'none'; base-uri 'self'.`
- Proteger rotas públicas sensíveis: `/api/contratos/pdf` e `/api/financeiro/asaas-balance` exigir `requireAuth` ou token efêmero `?token=JWT(slug, exp 5min)`. `/api/financeiro/seed.ts` exigir `requireAdmin` ou remover de prod.
- Validar HMAC em webhooks Next.js: ler `asaas-access-token / X-Assinafy-Signature` e comparar com `hash_hmac('sha256', rawBody, secret)` como já faz a versão PHP.

P3 — Alta — até 14 dias

- Implementar **isolamento por posse** (IDOR): adicionar coluna `criado_por / tenant_id` em propostas, contratos, lançamentos, orcamentos, clientes + migração; criar helper `requireOwnership(table, id, user.id)` que faz `SELECT tenant_id WHERE id=?` e compara; filtrar toda listagem com `WHERE tenant_id=$user.id`. Como ERP é single-tenant hoje, ao menos restringir por nível e auditar log de acesso.
- RBAC real: estender `requireAdmin` para `requireRole('financeiro', 'comercial')` e aplicar em `financeiro/*`, `configuracoes/asaas.ts`, `sistema/usuarios.php`. Frontend Sidebar deve espelhar papéis do backend.
- Criptografar colunas `configuracao_empresa.*_api_key` com `pgcrypto / AES-256-GCM` e descriptografar apenas em memória.

P4 — Média/Baixa — até 30 dias

- Corrigir CORS: `vercel.json` não combinar `Allow-Origin: *` com `Allow-Credentials: true`; restringir a `https://*.vercel.app`, `https://wedistinto.com` e validar `Origin` no `helpers.ts` `OPTIONS`.
- Substituir `innerHTML = 'literal'` por `textContent` onde não há HTML; adicionar ESLint rule `no-dangerous-innerhtml + no-new-func`.
- Adicionar rate-limit e CAPTCHA em rotas públicas (`/api/propostas/publica`, `orcamentos/solicitar`) e tokens de aprovação com expiração.
- Implementar auditoria: `logar user.id + IP + id` afetado em `propostas_historico` em todo `POST/PUT/DELETE`.

Issues para o GitHub — pronto para copiar e colar

Cada bloco entre --- **ISSUE n** --- é o corpo completo de uma issue em Markdown. Cole no GitHub com título e labels indicados. Achados triviais relacionados foram agrupados para evitar spam.

--- ISSUE 1 ---

Título: [Segurança] JWT_SECRET e credenciais MySQL com fallback hardcoded (bypass se env ausente)

Labels sugeridas: security, crítica

Descrição do problema e por que é explorável

O código usa `|| 'valor-produção'` como fallback para segredos críticos. Se a Vercel não tiver a variável de ambiente setada, a aplicação sobe com segredo público versionado no git, permitindo forjar JWT e conectar no banco de produção.

Evidência — arquivo:linha com trecho

```
src/lib/auth.ts:5 const JWT_SECRET = process.env.JWT_SECRET || 'erp_distinto_jwt_secret_key_2026_vercel'
src/lib/db.ts:7-11 host='srv952.hstgr.io' user='u306254544_poncem' password='!@Jeane&w;#1'
```

Repetido em `scratch/import_existing_contracts.js:7-11` e `scratch/seed_igor_gabriela_contract.js:5-9`. Nenhum throw se default (grep `throw.*JWT_SECRET → 0`).

Impacto

Forja de `distinto_token` (30 dias) → autenticação como qualquer usuário; conexão direta no MySQL Hostinger com credencial vazada.

Sugestão de correção

Trocar fallback por `if (!process.env.JWT_SECRET) throw new Error('JWT_SECRET missing')` (e idem para `MYSQL_*`). Falhar no build/start se ausente. Setar secrets em Vercel env (Production + Preview). Rotacionar JWT secret e senha MySQL imediatamente. Remover `scratch/` com credenciais do repo e limpar histórico com `git filter-repo`.

Critérios de aceite

- [] App não inicia se `JWT_SECRET/MYSQL_PASSWORD` ausentes (erro explícito no log)
- [] Vercel env contém `JWT_SECRET` e `MYSQL_*` (verificado em dashboard)
- [] Novo JWT invalida tokens antigos (usuários precisam relogar)
- [] Senha MySQL rotacionada e testada
- [] Histórico git limpo (`git log -p` não mostra `!@Jeane`)

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] Segredos reais versionados em `scratch/` (Gemini + Neon) no histórico git

Labels sugeridas: security, crítica

Descrição do problema e por que é explorável

Chaves de API reais estão em arquivos de debug versionados, expostas a qualquer pessoa com acesso ao repo e no histórico git mesmo após deletar o arquivo.

Evidência — arquivo:linha com trecho

```
scratch/test_env_key.py:6 key='AlzaSyCEK588doDrGt5myWeF3BmfRDd4o_aXTaA' (Gemini)
scratch/check_db.py:9 password='npg_3fXdCHMbS2xJ' + find_proposal.py:8, test_gemini.py:11, update_db_key.py:8 (Neon). git log -p --all -S
npg_ confirma desde commit inicial.
```

Impacto

Uso indevido da conta Google Gemini (custos) e acesso total ao Postgres Neon (leitura/escrita de `configuracao_empresa` com todas as API keys).

Sugestão de correção

Rotacionar Gemini key e Neon password imediatamente. Mover para `process.env.GEMINI_API_KEY / NEON_PASSWORD` sem default. Remover `scratch/*.py` com segredos do repo, adicionar `scratch/` ao `.gitignore`, limpar histórico (BFG `--delete-files check_db.py` ou `filter-repo`), `force-push` e orientar re-clone.

Critérios de aceite

- [] Chaves antigas revogadas no console Google/Neon
- [] Novas chaves apenas em env (não em arquivo)
- [] `git log -p` não retorna `Alza/npg_`
- [] `scratch/` ignorado no `.gitignore`

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Bypass de autenticação em ambiente não-produção + seed admin default exposto

Labels sugeridas: security, alta

Descrição do problema e por que é explorável

Fallback de dev autentica como admin sem token; arquivo de seed com senha default fica acessível via HTTP.

Evidência — arquivo:linha com trecho

```
src/lib/auth.ts:56-62 if (process.env.NODE_ENV !== 'production') return {id:'faustinosdg@gmail.com', nivel:1}
setup/seed.php:11 $senha='Distinto@2026' $email='admin@distinto.com.br' sem exigirAutenticacao(), acessível em /setup/seed.php.
```

Impacto

Bypass total em preview/staging ou build local; recriação de admin com senha conhecida se `setup/seed.php` for chamado.

Sugestão de correção

Condicionar dev bypass a `ALLOW_DEV_AUTH===true` explícito, nunca em preview. Proteger `setup/seed.php` com `exigirAdmin()` ou removê-lo do deploy (bloquear via `.htaccess/vercel.json`). Forçar troca de senha no primeiro login.

Critérios de aceite

- [] Requisição sem cookie em `NODE_ENV=production` retorna 401 (não 200)

- [] /setup/seed.php retorna 403 sem login admin ou 404 em prod
- [] Senha Distinto@2026 não funciona após rotação

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] Stored XSS público via dangerouslySetInnerHTML + new Function (propostas/contratos)

Labels sugeridas: security, crítica

Descrição do problema e por que é explorável

Dados de propostas (controlados por usuário autenticado) são renderizados sem sanitização em página pública /p/[slug] via dangerouslySetInnerHTML e executados via new Function, permitindo roubo de sessão de visitantes e admin que visualiza a proposta.

Evidência — arquivo:linha com trecho

```
src/pages/p/[slug].tsx:233 dangerouslySetInnerHTML=({__html: html}) (html = renderCasamento + planModalHtml)
src/components/ContratosView.tsx:1024 dangerouslySetInnerHTML contrato_texto
src/components/ModelosContratoView.tsx:734 codigoHtmlRaw
src/lib/propostas/scripts.ts:134 div.innerHTML=`${nome}` sem esc()
src/pages/p/[slug].tsx:267 new Function(code)()
```

Impacto

Atacante autenticado salva payload em dados_json (ex: nome_noivo=) → todo visitante de /p/[slug] executa. Também afeta preview admin. Sem DOMPurify (package.json sem dependência) e sem CSP.

Sugestão de correção

Instalar isomorphic-dompurify e sanitizar antes de injetar. Garantir esc() em TODAS as interpolações de templates/casamento.ts. Trocar scripts.ts:134 innerHTML nome por esc(nome) ou textContent. Remover new Function (usar componentes React). Adicionar CSP script-src 'self' sem unsafe-eval.

CrITÉrios de aceite

- [] Payload <script> em nome_noivo é escapado no HTML (ver fonte)
- [] DOMPurify chamado antes de cada dangerouslySetInnerHTML (grep)
- [] new Function removido (grep 0)
- [] CSP header presente em vercel.json

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

Título: [Segurança] IDOR massivo + ausência de isolamento (qualquer autenticado acessa tudo) e RBAC só no frontend

Labels sugeridas: security, alta

Descrição do problema e por que é explorável

Nenhuma tabela tem coluna user_id/tenant_id; todas as listagens retornam dados globais e todo UPDATE/DELETE aceita ID arbitrário sem checar posse. RBAC é só visual na Sidebar; backend valida nivel==1 em apenas 5 rotas.

Evidência — arquivo:linha com trecho

```
src/lib/db.ts:31-289 initTables() sem user_id
src/pages/api/comercial/propostas.ts: SELECT * FROM propostas
api/financeiro/lancamentos.php: PUT/DELETE WHERE id=? sem AND user_id
src/pages/api/configuracoes/asaas.ts: requireAuth (não requireAdmin) → qualquer autenticado altera asaas_api_key global
src/components/Sidebar.tsx:12 todos os tabs visíveis para nivel 0
```

Impacto

Comprometimento de conta de baixo privilégio → acesso/alteração de todos os lançamentos financeiros, contratos e propostas; alteração de credencial Asaas global.

Sugestão de correção

Adicionar coluna criado_por/tenant_id + migração; criar middleware requireOwnership(table,id,userId) e filtrar listagens por tenant. Estender requireAdmin/requireRole para financeiro e configurações; espelhar gates no Sidebar via user.nivel do backend.

CrITÉrios de aceite

- [] Listagem com usuário A não retorna dados criados por B (teste com 2 contas)
- [] PUT /api/financeiro/lancamentos com id de outro retorna 403/404
- [] POST /api/configuracoes/asaas com nivel 0 retorna 403
- [] Sidebar oculta Financeiro para nivel 0

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

Título: [Segurança] Rotas e webhooks públicos expõem dados sensíveis sem auth/HMAC

Labels sugeridas: security, alta

Descrição do problema e por que é explorável

Contratos, saldo Asaas e seed são acessíveis sem login; webhooks Next.js aceitam qualquer POST e mutam lançamentos, sem validar assinatura.

Evidência — arquivo:linha com trecho

```
src/pages/api/contratos/pdf.ts:5 handler sem requireAuth → SELECT WHERE id=$1
src/pages/api/financeiro/asaas-balance.ts:5 sem auth → getBalanceAndExtract()
```


src/pages/api/financeiro/seed.ts:5 sem auth → INSERT clientes/lancamentos

src/pages/api/financeiro/webhook-asaas.ts:6 sem HMAC → INSERT lancamentos (PHP valida: api/financeiro/webhook-asaas.php:47 asaas-access-token)

Impacto

Enumeração de contratos por id/slug; vazamento de saldo/cobranças Asaas; injeção de lançamentos falsos via webhook forjado; criação de dados fake via seed.

Sugestão de correção

Exigir requireAuth/requireAdmin ou token efêmero (JWT slug+exp) em pdf/balance/seed. Replicar validação HMAC do PHP no handler Next.js (comparar header com configuracao_empresa.asaas_webhook_token).

CrITÉRIOS de aceite

- [] GET /api/contratos/pdf?id=X sem cookie retorna 401
- [] GET /api/financeiro/asaas-balance sem auth retorna 401
- [] POST /api/financeiro/webhook-asaas sem token válido não altera lancamentos
- [] /api/financeiro/seed exige admin

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

Título: [Segurança] Hardening: CSP ausente e CORS com Allow-Origin * + Credentials

Labels sugeridas: security, média

Descrição do problema e por que é explorável

Sem Content-Security-Policy, XSS tem impacto máximo. CORS com * + credentials é mal configurado e indica intenção de liberar tudo.

Evidência — arquivo:linha com trecho

vercel.json:28-37 headers: Access-Control-Allow-Origin: * + Allow-Credentials: true, sem Content-Security-Policy

src/lib/helpers.ts:17 res.setHeader('Access-Control-Allow-Origin', '*') em OPTIONS.

Impacto

Amplifica XSS (sem CSP para bloquear inline) e pode permitir requisições cross-origin indesejadas se credentials forem usados.

Sugestão de correção

Adicionar CSP em vercel.json (default-src 'self'; script-src 'self' https://fonts.googleapis.com; object-src 'none'). Restringir CORS a domínios específicos (wedistinto.com, vercel.app) e não usar * com credentials.

CrITÉRIOS de aceite

- [] Response headers contém Content-Security-Policy
- [] OPTIONS com Origin estranha não retorna Allow-Origin: *
- [] CSP bloqueia inline não autorizado (teste com script inline)

--- FIM ISSUE 7 ---

Fim do relatório. Para regerar: python docs/security-audit/generate_report.py. Verifique também docs/security-audit/_chart_sev.png e _chart_cat.png (gerados automaticamente).